TI202 - Algorithmique et Structures de données 1

TP8 - Listes chaînées (2h)

Échauffement

Exercice 1. Reprenez la petite bibliothèque de fonctions décrite dans l'exercice 3 du TD9.

(a) Dans un fichier `list_utils.h`, déclarez les types `t_cell` et `t_list`, puis copiez les prototypes de toutes les fonctions.

(b) Dans un fichier `list_utils.c`, implémentez toutes les fonctions.

(c) Dans un fichier `main.c`, écrivez le programme de l'exercice 4 du TD9.

Renforcement

Exercice 2. Dans cet exercice, on souhaite représenter un mot par une liste chaînée de lettres. Chaque lettre sera représentée par un caractère et constituera une cellule de la liste. Dans toute la suite de l'exercice, la définition des types structurés et l'en-tête des fonctions seront à placer dans un fichier `word.h` et les implémentations des fonctions dans un fichier `word.c`. Le programme principal sera dans un fichier `main.c`.

1. (a) Définir le type `t_letter` représentant une lettre. Ce type constitue une cellule de la liste chaînée.
   (b) Définir le type `t_word` représentant un mot. Ce type est une liste chaînée de lettres.
2. (a) Écrire une fonction `t_letter * init_letter(char c)` qui alloue dynamiquement une lettre et l'initialise avec le caractère `c`. Cette fonction renvoie un pointeur sur la lettre allouée.
   (b) Écrire une fonction `void free_letter(t_letter * l)` qui libère la mémoire allouée pour la lettre `l`.
   (c) Écrire une fonction `t_word init_word()` qui initialise et renvoie un mot vide.
3. (a) Écrire une fonction `void prepend_letter(t_word * w, char c)` qui ajoute la lettre de valeur `c` au début du mot `w`.
   (b) Écrire une fonction `void append_letter(t_word * w, char c)` qui ajoute la lettre de valeur `c` à la fin du mot `w`.
   (c) Écrire une fonction `t_letter * pop_letter(t_word * w)` qui renvoie la première lettre du mot `w` et la retire du mot.
   (d) Écrire une fonction `void free_word(t_word * w)` qui libère la mémoire allouée pour le mot `w`.
   (e) Écrire une fonction `void display_word(t_word * w)` qui affiche le mot `w` sur la sortie standard. Cette fonction affichera toutes les lettres du mot sur une seule ligne.
4. (a) Écrire une fonction `t_word read_word(char * string)` qui renvoie un mot contenant les lettres de la chaîne de caractères `string`. Les lettres du mot seront ajoutées dans l'ordre de la chaîne de caractères.
   (b) Écrire une fonction `t_word copy_word(t_word w)` qui initialise et renvoie une copie du mot `w`. Cette copie est un nouveau mot contenant les mêmes lettres que `w`.

(c) Écrire une fonction `void concat_words(t_word * w1, t_word * w2)` qui concatène les mots `w1` et `w2`. Le mot `w2` est ajouté à la fin du mot `w1`.

Exemple : *Si* `w1` *est la liste chaînée représentant* `"Howdy"` *et* `w2` *la liste chaînée représentant* `" partner"`*, alors après l'appel de la fonction* `concat_words(w1, w2)`*, le mot* `w1` *représentera* `"Howdy partner"`*.*

(d) Écrire une fonction `t_word reverse_word(t_word w)` qui initialise et renvoie le mot inverse du mot `w`. Le mot `w` ne doit pas être modifié.

Approfondissement

Exercice 3. [Ouroboros (Prologin 2024)]

Jörmungandr est un serpent légendaire de la mythologie nordique qui entoure tout Midgard (*la terre*), en passant par certaines villes, jusqu'à se mordre sa queue. Les villes rencontrées en partant de la queue de Jörmungandr jusqu'à sa tête sont données en entrée dans une liste de chaînes de caractères. Sa tête — et donc sa queue, qui se situent au même endroit car il se mord la queue — se trouvent toujours entre deux villes. Initialement, elles se situent entre la dernière ville listée et la première (souvenez-vous que la liste décrit un cycle), et Jörmungandr avance de gauche à droite dans la liste.

Dans M années aura lieu le Ragnarök. Chaque année jusqu'au Ragnarök, Jörmungandr prend l'une des actions suivantes :

- A: Avancer d'une ville de manière circulaire dans la liste ;
- M: Manger la ville la plus proche devant sa tête ;
- R: Se retourner, ce qui signifie qu'il avancera alors dans la direction opposée par la suite. S'il avançait de gauche à droite, il avance alors de droite à gauche, et inversement ;
- C: Recracher la dernière ville qu'il a mangé et qu'il n'a pas encore recraché, pour la placer devant sa tête.

Notez donc que si, après s'être retourné, Jörmungandr se dirige de droite à gauche dans la liste, il mangerait alors la ville située immédiatement à gauche de sa tête.

On vous indiquera les actions de Jörmungandr pour les M prochaines années. Votre tâche est de prédire l'état final des villes lors du Ragnarök, en les listant dans l'ordre en partant de sa queue jusqu'à sa tête.

Entrée L'entrée contiendra:

- Sur la première ligne, un entier N représentant le nombre de villes dans la liste.
- Sur la ligne suivante, un entier M, représentant le nombre d'années avant le Ragnarök.
- Sur les lignes suivantes, une liste de N chaînes de caractères, chacune représentant une ville: en partant de la queue de Jörmungandr. (Chaque chaîne de caractères est une chaîne non-vide de 10 caractères ou moins)
- Sur la dernière ligne, une chaîne de M caractères représentant les actions de Jörmungandr pour les M prochaines années.

Sortie Afficher, sur une ligne par ville, la liste des villes qui seront rencontrées lors du Ragnarök, dans l'ordre, en partant de la queue de Jörmungandr jusqu'à sa tête.

Exemple d'entrée

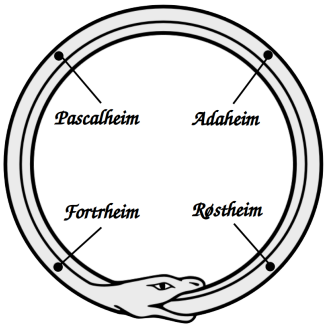
```
4
5
Rostheim
Adaheim
Pascalheim
Fortrheim
AMRAC
```

Exemple de sortie

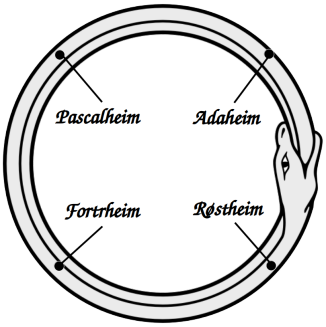
```
Adaheim
Fortrheim
Pascalheim
Rostheim
```

Commentaires

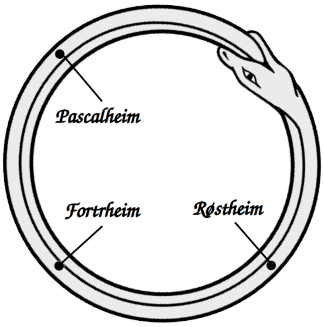
Situation initiale La situation décrit quatre villes autour de Midgard par lesquelles passe Jörmungandr : Rostheim, Adaheim, Pascalheim et Fortrheim. Initialement, la tête et la queue de Jörmungandr se situent entre les villes de Fortrheim et de Rostheim. Le Ragnarök se déroulera dans 5 ans, alors, Jörmungandr aura le temps d'effectuer 5 actions, qui sont les suivantes : Avancer, Manger, Se retourner, Avancer, Recracher.



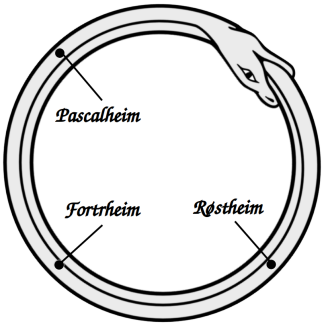
Année 1 Il commencera par avancer d'une ville, ce qui le placera entre les villes de Rostheim et Adaheim.



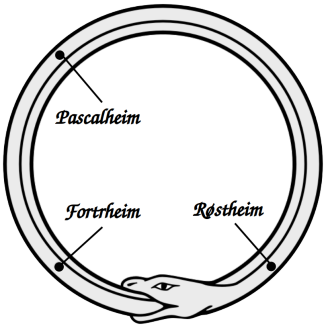
Année 2 Il mangera la ville la plus proche devant sa tête, Adaheim, alors sa tête se situera entre les villes de Rostheim et Pascalheim.



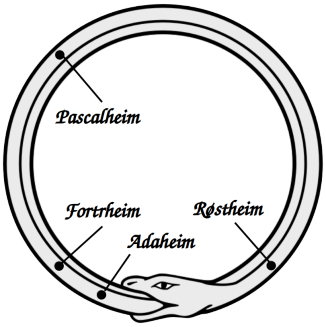
Année 3 Il se retournera. Sa tête se dirigera alors vers Rostheim.



Année 4 Ensuite, il avancera d'une ville, cette fois-ci dans l'autre sens car il s'est retourné. Sa tête se situera alors entre Rostheim et Fortrheim.



Année 5 Enfin, il recrachera la dernière ville qu'il a ingéré, c'est à dire Adaheim, juste en face de sa tête, c'est à dire entre Rostheim et Fortrheim. Alors, au moment du Ragnarök, sa tête se situera entre Rostheim et Adaheim.



En partant de sa queue et en listant les villes rencontrées le long de son corps, on obtiendrait alors : Adaheim, Fortrheim, Pascalheim, Rostheim.

Indications

- On peut penser à une liste chaînée pour représenter la liste des villes : chaque ville correspondra à une cellule, et pour simplifier les opérations, chaque ville pointera à la fois vers la ville suivante et la ville précédente.
- Dans le type liste, on peut choisir de stocker à la fois la première et la dernière ville pour simplifier les opérations de déplacement. On peut également encapsuler dans la liste l'orientation actuelle du serpent. Il est important de se rappeler que la liste est circulaire, de ce fait, la ville suivante de la dernière ville est la première ville, et la ville précédente de la première ville est la dernière ville.
- On peut également utiliser une liste chaînée (classique) pour stocker l'ensemble des villes mangées par Jörmungandr (pour les recracher plus tard).En effet, recracher une ville revient à retirer la tête de liste des villes mangées puis à l'ajouter à la liste des villes à visiter.